Booleans

Introduction to Computer Programming

Chandra Gummaluru

chandra-gummaluru.github.io

Type **bool** has exactly two values: `True` and `False`. They are the answers to yes-or-no questions.

```
>>> type(True)
<class 'bool'>
>>> 4 > 3
True
>>> 7 == 8
False
```

`True` and `False` are values, like `3` or `'hi'`, and are written without quotes.

A **comparison operator** compares two values and produces a `bool`.

operator	symbol	example	value
less than	`<`	`3 < 4`	`True`
greater than	`>`	`3 > 8`	`False`
equal to	`==`	`7 == 7`	`True`
greater than or equal to	`>=`	`3.5 >= 3.4`	`True`
less than or equal to	`<=`	`3.5 <= 3.4`	`False`
not equal to	`!=`	`4 != 4`	`False`

Be careful: = assigns a value to a variable, while == asks whether two values are equal.

Comparisons work with variables and expressions. Arithmetic happens before comparison.

```
>>> day = 23
>>> day < 30
True
>>> day + 1 > 24
False
>>> day == 23.0
True
```

```
day + 1 > 24

(day + 1) > 24

24 > 24

False
```

An `int` and a `float` are compared by value, so `4 == 4.0` is `True`.

Thumbs up or down: what value does the last line produce?

```
1  day = 28
2  day + 1 > 20
```

- ✓ True
- B False

What value does this produce?

```
1  3 + 1 != 4.0
```

A True

✓ False

logical operators

The **logical operators** `not`, `and` and `or` combine `bool` values.

a	b	a and b	a or b
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

a	not a
True	False
False	True

- `a and b` is `True` only when both are `True`
- `a or b` is `True` when at least one is `True`

`not` has higher precedence than `and`, which has higher precedence than `or`. All three come after arithmetic and comparisons.

```
monday = True
rainy = False
sunny = not rainy
```

expression	value
`not rainy`	`True`
`not not rainy`	`False`
`monday and rainy`	`False`
`monday or rainy`	`True`

What value does the last line produce?

```
1  monday = True
2  rainy = False
3  sunny = not rainy
4  not monday or sunny
```

- ✓ True
- B False

What value does the last line produce?

```
1  monday = True
2  rainy = False
3  sunny = not rainy
4  not (monday or sunny)
```

A True

✓ False

Python stops evaluating and/or as soon as it knows the answer. This is called **short-circuit** (or **lazy**) evaluation.

or

`True or anything is True`

```
>>> 3 < 4 or 1 / 0 == 5
True
```

and

`False and anything is False`

```
>>> 7 < 3 and 1 / 0 == 5
False
```

In both cases the right-hand side is never evaluated, so the division by zero never happens.

The order matters. If the left side could crash, put a **guard** in front of it.

crashes

```
>>> n = 0
>>> total = 100
>>> total / n > 75
ZeroDivisionError
```

guarded

```
>>> n = 0
>>> total = 100
>>> n != 0 and total / n > 75
False
```

What happens when the last line runs?

```
1  n = 0
2  total = 100
3  total > 50 and n != 0 and total / n > 75
```

A `True`

✓ `False`

C `ZeroDivisionError`

What happens when the last line runs?

```
1  n = 0
2  total = 100
3  total / n > 75 or n == 0
```

A `True`

B `False`

✓ `ZeroDivisionError`

testing strings

Strings are compared in **dictionary order**, one character at a time.

operator	example	value
`<`	`'Jen' < 'Paul'`	`True`
`>`	`'Jen' > 'Paul'`	`False`
`==`	`'A' == 'a'`	`False`
`>=`	`'a' >= 'a'`	`True`
`!=`	`'apple' != 'apple '`	`True`

- uppercase and lowercase letters are different characters
- == works between any types; `'a' < 3` is a `TypeError`

What value does this produce?

```
1  'John' < 'Jake'
```

A True

✓ False

What value does this produce?

```
1  'John' < 'Johnny'
```

- ✓ True
- B False

The **in** operator checks whether one string appears inside another, and produces a `bool`.

```
>>> 'Jen' in 'Jennifer'
True
>>> 'jen' in 'Jennifer'
False
>>> 'nif' in 'Jennifer'
True
>>> '' in 'Jennifer'
True
```

The characters must appear together and in order, and the empty string is in every string.

Which expression produces `False`?

```
1  lyrics = 'abc easy as 123'
```

A `'easy' in lyrics`

B `str(len('mj')) in lyrics`

✓ `'cab' in lyrics`

D `'' in lyrics`

docstring_exercise.ipynb

Complete the example calls and then write a docstring description for each function. This problem is about **reading and understanding code**, not just writing it.

(a)

In [ ]

```
def function_a(s1: str, s2: str) -> bool:
    """Return True iff s1 and s2 are the same string.

    >>> function_a('cat', 'cat')
    True
    >>> function_a('cat', 'cats')
    False
    """
    return s1 in s2 and s2 in s1
```

(b)

In [ ]

```python
def function_b(s1: str, s2: str) -> bool:
    """Return True iff s1 and s2 have the same first character and
    s1 and s2 have the same last character.

    Precondition: len(s1) > 0 and len(s2) > 0

    >>> function_b('cat', 'coat')
    True
    >>> function_b('cat', 'dog')
    False
    """
    return s1[0] == s2[0] and s1[-1] == s2[-1]
```

(c)

In [ ]

```python
def function_c(s1: str, s2: str) -> bool:
    """Return True iff the length of s1 is less than or equal to
    the length of s2.

    >>> function_c('hi', 'hello')
    True
    >>> function_c('hello', 'hi')
    False
    """
    return min(len(s1), len(s2)) == len(s1)
```

(d)

In [ ]

```python
def function_d(s1: str, s2: str) -> bool:
    """Return True iff the first and last characters in s1 are the same,
    the first and last characters in s2 are the same, or both.

    Precondition: len(s1) > 0 and len(s2) > 0

    >>> function_d('pop', 'cat')
    True
    >>> function_d('cat', 'dog')
    False
    """
    return s1[0] == s1[-1] or s2[0] == s2[-1]
```

(e)

In [ ]

```python
def function_e(s1: str, s2: str) -> bool:
    """Return True iff the number representing the length of s1 occurs in s2.

    >>> function_e('abc', 'CSC 3')
    True
    >>> function_e('abc', 'CSC108')
    False
    """
    return str(len(s1)) in s2
```

(i) Match each docstring description to the function it describes. Note that "iff" stands for "if and only if".

1. Return True iff s1 and s2 have the same first character and s1 and s2 have the same last character.
2. Return True iff the number representing the length of s1 occurs in s2.
3. Return True iff the first and last characters in s1 are the same, the first and last characters in s2 are the same, or both.
4. Return True iff s1 and s2 are the same string.
5. Return True iff the length of s1 is less than or equal to the length of s2.

(ii) Which function(s) also need a precondition? Add the precondition(s).

(i) 1 → `function_b`, 2 → `function_e`, 3 → `function_d`, 4 → `function_a`, 5 → `function_c`.

```
                                                                    len(s1) > 0
and len(s2) > 0
```

Which function does this description match?

Return True if and only if s1 and s2 have the same first character and s1 and s2 have the same last character.

A `function_a`

✓ `function_b`

C `function_c`

D `function_d`

E `function_e`

Which function does this description match?

Return True if and only if the number representing the length of s1 occurs in s2.

A `function_a`

B `function_b`

C `function_c`

D `function_d`

✓ `function_e`

Which function does this description match?

Return True if and only if the first and last characters in s1 are the same, the first and last characters in s2 are the same, or both.

A `function_a`

B `function_b`

C `function_c`

✓ `function_d`

E `function_e`

Which function does this description match?

Return True if and only if s1 and s2 are the same string.

- ✓ `function_a`
- B `function_b`
- C `function_c`
- D `function_d`
- E `function_e`

Which function does this description match?

Return True if and only if the length of s1 is less than or equal to the length of s2.

A
```
function_a
```

B
```
function_b
```

✓
```
function_c
```

D
```
function_d
```

E
```
function_e
```

Which of `function_a` to `function_e` also need a precondition?

A `function_b` only

B `function_d` only

✓ `function_b` and `function_d`

D `function_c` and `function_e`

Some string methods **test** a string and produce a `bool`. The `is` methods check every character.

method	example	value
`isupper`	`'CSC108'.isupper()`	`True`
`islower`	`'Hello'.islower()`	`False`
`isdigit`	`'2026'.isdigit()`	`True`
`isalpha`	`'CSC108'.isalpha()`	`False`
`startswith`	`'hello'.startswith('he')`	`True`
`endswith`	`'hello.py'.endswith('.py')`	`True`

`isupper` and `islower` ignore characters that aren't letters, but the string needs at least one letter.

This expression produces `True`.

```
1  '2026!'.isdigit()
```

A True

✓ False

testing_strings.py

Program

```
1
```

Shell

```
>>> 'Jen' in 'Jennifer'
True
>>> 'jen' in 'Jennifer'
False
>>> 'CSC108'.isupper()
True
>>> '2026'.isdigit()
True
>>> 'hello'.startswith('he')
True
>>> 'hello.py'.endswith('.py')
True
```